



مبانی برنامه نویسی

Structured program development

Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>

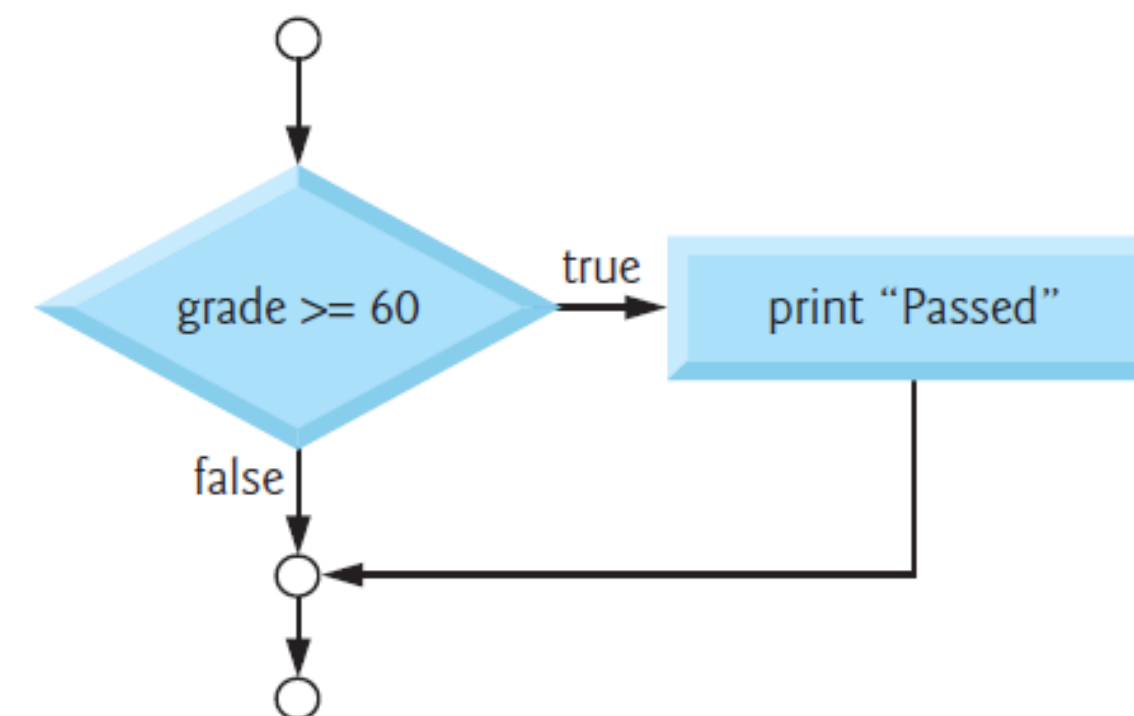


https://github.com/safayani/Programming_Basics_course



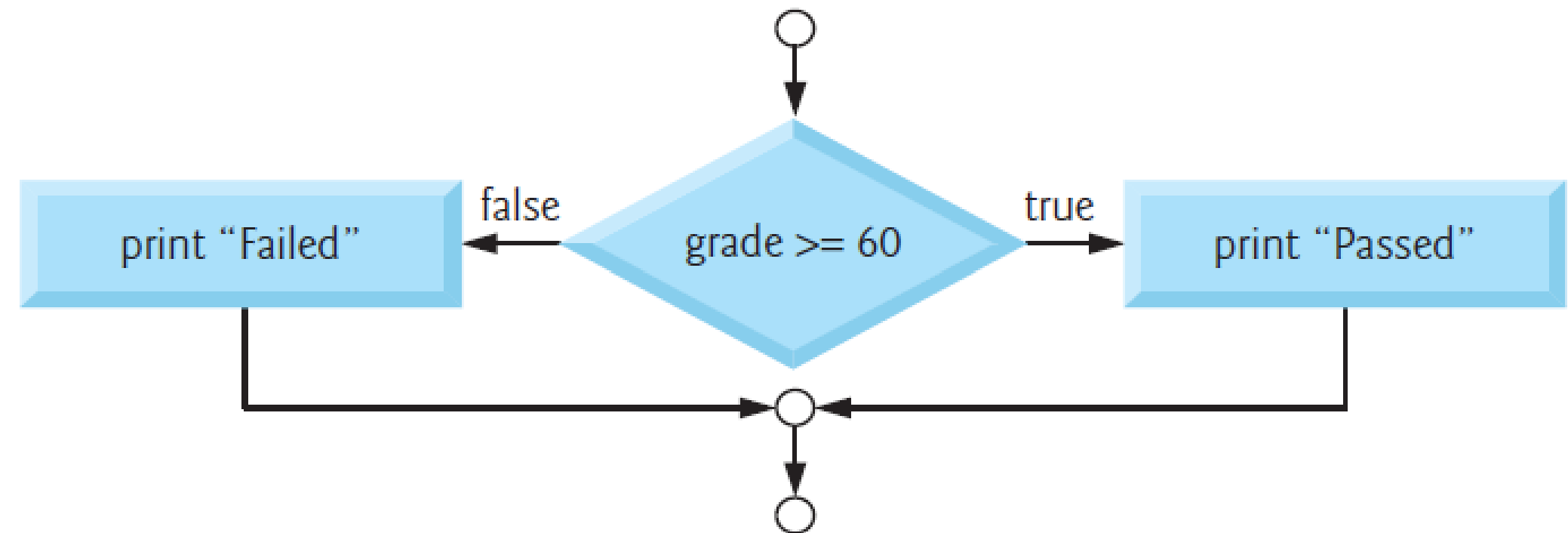
The if Selection Statement

```
if (grade >= 60) {  
    puts("Passed");  
} // end if
```



The if...else Selection Statement

```
if (grade >= 60) {  
    puts("Passed");  
} // end if  
else {  
    puts("Failed");  
} // end else
```



conditional operator (?:)

```
puts((grade >= 60) ? "Passed" : "Failed");
```

- The **conditional operator** ?: is C's only ternary operator, taking three operands.
- Syntax: condition ? value_if_true : value_if_false
- It returns a value and can be used directly in expressions and function arguments.
- Best practice: Ensure the second and third operands are of the **same type** to avoid errors.
- It provides a compact alternative to simple if...else statements.

Nested if...else Statements

If student's grade is greater than or equal to 90

Print "A"

else

If student's grade is greater than or equal to 80

Print "B"

else

If student's grade is greater than or equal to 70

Print "C"

else

If student's grade is greater than or equal to 60

Print "D"

else

Print "F"


```

if (grade >= 90) {
    puts("A");
} // end if
else {
    if (grade >= 80) {
        puts("B");
    } // end if
    else {
        if (grade >= 70) {
            puts("C");
        } // end if
        else {
            if (grade >= 60) {
                puts("D");
            } // end if
            else {
                puts("F");
            } // end else
        } // end else
    } // end else
} // end else

```

```

if (grade >= 90) {
    puts("A");
} // end if
else if (grade >= 80) {
    puts("B");
} // end else if
else if (grade >= 70) {
    puts("C");
} // end else if
else if (grade >= 60) {
    puts("D");
} // end else if
else {
    puts("F");
} // end else

```

Blocks and Compound Statements

- Always use **braces** { } for the bodies of if and else statements, even for **single statements**.
- This prevents logic errors, such as the "**dangling else**" problem, where an else is mistakenly associated with the wrong if.
- Using braces ensures the code's structure is clear and the logic is executed as intended.

-

The “Dangling Else” Problem

Original Code	Compiler Interpretation
<pre>int x = 10; int y = 5; // The code looks like this: if (x > 5) if (y > 10) printf("x and y are both large\n"); else // This 'else' seems to belong to 'if (x > 5)' printf("x is 5 or less\n");</pre>	<pre>// How the compiler actually interprets the code: if (x > 5) { if (y > 10) { printf("x and y are both large\n"); } else { // The "dangling else" attaches to the INNER if! printf("x is 5 or less\n"); } }</pre>
Expected Output: Nothing. Real Output: "x is 5 or less"	

The Solution - Always Use Braces

```
int x = 10;
int y = 5;

// The logic is now clear and correct:
if (x > 5) {
    if (y > 10) {
        printf("x and y are both large\n");
    }
} else { // This 'else' is now correctly paired with the OUTER 'if'
    printf("x is 5 or less\n");
}
```

- **Correct Logic:** The if ($x > 5$) condition is true, so the outer block is executed. The inner if ($y > 10$) is false, and since there is no else attached to it, nothing happens. The final else block is correctly skipped.

Empty Statement

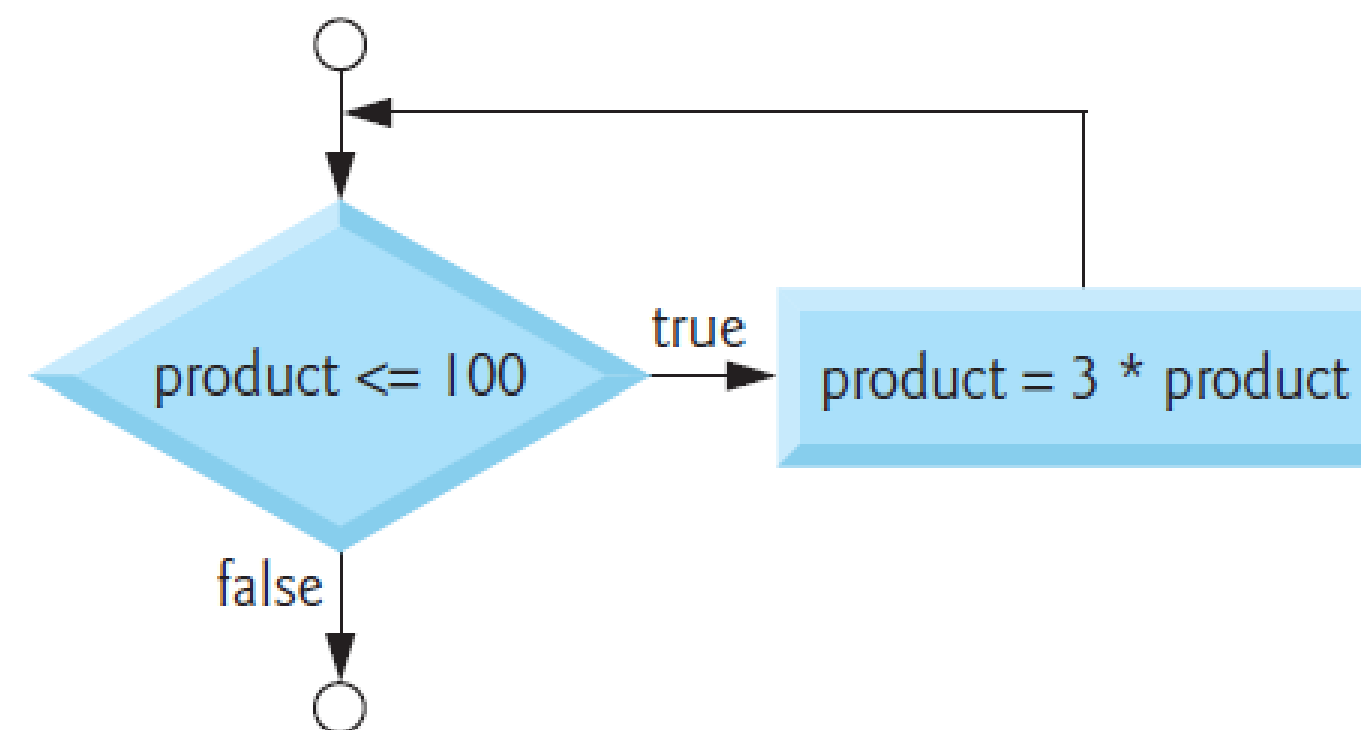
```
if (grade >= 60);
```

- leads to a logic error in single-selection if statements and a syntax error in double selection and nested if...else statements.

The while Iteration Statement

- Calculating the First Power of 3 Greater Than 100

```
int product = 3;  
while (product <= 100) {  
    product = 3 * product;  
}
```



```
int product = 3;
while (product <= 100) {
    product = 3 * product;
}
```

The while Iteration Statement

- **Calculating the First Power of 3 Greater Than 100**
- The while **loop** repeats as long as its condition remains true.
- The loop's **body** (a single or compound statement) must modify a variable so the condition can eventually become false.
- If the condition never becomes false, an **infinite loop** occurs, which is a logic error.
- When the condition evaluates to false, the loop terminates, and the program continues with the next statement.

Counter- Controlled Iteration

- A class of ten students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Determine the class average on the quiz.*

```
1 // fig03_02.c
2 // Class average program with counter-controlled iteration.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main(void) {
7     // initialization phase
8     int total = 0; // initialize total of grades to 0
9     int counter = 1; // number of the grade to be entered next
10
11     // processing phase
12     while (counter <= 10) { // loop 10 times
13         printf("%s", "Enter grade: "); // prompt for input
14         int grade = 0; // grade value
15         scanf("%d", &grade); // read grade from user
16         total = total + grade; // add grade to total
17         counter = counter + 1; // increment counter
18     } // end while
19
20     // termination phase
21     int average = total / 10; // integer division
22     printf("Class average is %d\n", average); // display result
23 }
```

```
Enter grade: 98
Enter grade: 76
Enter grade: 71
Enter grade: 87
Enter grade: 83
Enter grade: 90
Enter grade: 57
Enter grade: 79
Enter grade: 82
Enter grade: 94
Class average is 81
```


Some refinements: Sentinel-Controlled Iteration

- *Develop a class-averaging program that will process an arbitrary number of grades each time the program is run.*
- **Sentinel Values:** a **sentinel value** to indicate “end of data entry.”
- Proper message in case of zero grade

```

6 int main(void) {
7     // initialization phase
8     int total = 0; // initialize total
9     int counter = 0; // initialize loop counter
10
11    // processing phase
12    // get first grade from user
13    printf("%s", "Enter grade, -1 to end: "); // prompt for input
14    int grade = 0; // grade value
15    scanf("%d", &grade); // read grade from user
16
17    // loop while sentinel value not yet read from user
18    while (grade != -1) {
19        total = total + grade; // add grade to total
20        counter = counter + 1; // increment counter
21
22        // get next grade from user
23        printf("%s", "Enter grade, -1 to end: "); // prompt for input
24        scanf("%d", &grade); // read next grade
25    } // end while
26
27    // termination phase
28    // if user entered at least one grade
29    if (counter != 0) {
30
31        // calculate average of all grades entered
32        double average = (double) total / counter; // avoid truncation
33
34        // display average with two digits of precision
35        printf("Class average is %.2f\n", average);
36    } // end if
37    else { // if no grades were entered, output message
38        puts("No grades were entered");
39    } // end else
40 } // end function main

```

```

Enter grade, -1 to end: 75
Enter grade, -1 to end: 94
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 70
Enter grade, -1 to end: 64
Enter grade, -1 to end: 83
Enter grade, -1 to end: 89
Enter grade, -1 to end: -1
Class average is 82.50

```

```

Enter grade, -1 to end: -1
No grades were entered

```

Always Use Braces in a while Statement

```
while (grade != -1)
    total = total + grade; // add grade to total
    counter = counter + 1; // increment counter

    // get next grade from user
    printf("%s", "Enter grade, -1 to end: "); // prompt for input
    scanf("%d", &grade); // read next grade
```

ERR ⊗ This would cause an infinite loop if the user did not input -1 as the first grade.

Type Conversion for Accurate Division

- **Problem:** Integer division (int / int) truncates the fractional part.
- **Solution:** Use an **explicit cast** to convert operands:
 - (double) total creates a temporary floating-point value
- **Result:**
 - (double) total / counter forces **floating-point division**
 - The compiler performs **implicit conversion** on counter
- **Key Point:** Cast operator is a unary operator with high precedence, ensuring accurate arithmetic results. This precedence is one level higher than that of the multiplicative operators *, / and %.

Formatting and Considerations for Floating-Point Numbers

- **Precision Control:** Use %.nf in printf to specify decimal places (e.g., %.2f for two digits).
- **Rounding:** The displayed value is rounded, but the value in memory remains unchanged.
- **Inherent Imprecision:** Floating-point numbers are often approximations due to fixed memory allocation.
- **Key Implications:**
 - Avoid using == to compare floating-point numbers for equality.
 - They are suitable for most applications where exact precision is not critical.

```
#include <math.h>
#define TOLERANCE 0.00001

if (fabs(number1 - number2) < TOLERANCE) {
```

Title: Real Estate Exam Results Analyzer

- **Objective:**

- Create a program to summarize the exam results for 10 students from a real estate course.

- **Input Requirements:**

- The program will prompt the user to “Enter result” 10 times.
- Users will input either a 1 for Pass or a 2 for Fail for each student.

- **Program Logic & Output:**

- **Count:** Tally the total number of passes and fails.
- **Summarize:** Display the final counts (e.g., “Passed: 7, Failed: 3”).
- **Bonus Condition:** If more than 8 students passed, print the message: “Bonus to instructor!”

Pseudocode for examination-results problem

```
1  Initialize passes to zero
2  Initialize failures to zero
3  Initialize student to one
4
5  While student counter is less than or equal to ten
6      Input the next exam result
7
8      If the student passed
9          Add one to passes
10     else
11         Add one to failures
12
13     Add one to student counter
14
15 Print the number of passes
16 Print the number of failures
17 If more than eight students passed
18     Print "Bonus to instructor!"
```

```

6 int main(void) {
7     // initialize variables in definitions
8     int passes = 0;
9     int failures = 0;
10    int student = 1;

11
12    // process 10 students using counter-controlled loop
13    while (student <= 10) {
14        // prompt user for input and obtain value from user
15        printf("%s", "Enter result (1=pass,2=fail): ");
16        int result = 0; // one exam result
17        scanf("%d", &result);

18
19        // if result 1, increment passes
20        if (result == 1) {
21            passes = passes + 1;
22        } // end if
23        else { // otherwise, increment failures
24            failures = failures + 1;
25        } // end else

26
27        student = student + 1; // increment student counter
28    } // end while

29
30    // termination phase; display number of passes and fail
31    printf("Passed %d\n", passes);
32    printf("Failed %d\n", failures);

33
34    // if more than eight students passed, print "Bonus to
35    if (passes > 8) {
36        puts("Bonus to instructor!");
37    } // end if
38 } // end function main

```

```

Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 2
Enter Result (1=pass, 2=fail): 2
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 2
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 2
Passed 6
Failed 4

```

```

Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 2
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Enter Result (1=pass, 2=fail): 1
Passed 9
Failed 1
Bonus to instructor!

```

Fig. 3.6 | Analysis of examination results. (Part 2 of 2.)

Assignment Operators

Assignment operator	Sample expression	Explanation	Assigns
<i>Assume:</i> <code>int c = 3, d = 5, e = 4, f = 6, g = 12;</code>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 to f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 to g

Increment and Decrement Operators

Operator	Sample expression	Explanation
++	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

```

6  int main(void) {
7      // demonstrate postincrement
8      int c = 5; // assign 5 to c
9      printf("%d\n", c); // print 5
10     printf("%d\n", c++); // print 5 then postincrement
11     printf("%d\n\n", c); // print 6
12
13     // demonstrate preincrement
14     c = 5; // assign 5 to c
15     printf("%d\n", c); // print 5
16     printf("%d\n", ++c); // preincrement then print 6
17     printf("%d\n", c); // print 6
18 } // end function main

```

```

5
5
6

5
6
6

```


Operators	Grouping	Type
<code>++ (postfix) -- (postfix)</code>	right to left	postfix
<code>+ - (type) ++ (prefix) -- (prefix)</code>	right to left	unary
<code>* / %</code>	left to right	multiplicative
<code>+ -</code>	left to right	additive
<code>< <= > >=</code>	left to right	relational
<code>== !=</code>	left to right	equality
<code>?:</code>	right to left	conditional
<code>= += -= *= /= %=</code>	right to left	assignment

- `a = b = c = 5;`
- `num = 5; a = num++; // a=5; num=6`
- `x=1;y=2;                      x += y += 5; // x=8 y=7`

Max.C program that takes two numbers from the user and prints the maximum of the two

```
#include <stdio.h>

int main() {
    int num1, num2;
    printf("Enter the first number: ");
    scanf("%d", &num1);
    printf("Enter the second number: ");
    scanf("%d", &num2);
    if (num1 >= num2) { // Simplified condition
        printf("The maximum number is: %d\n", num1);
    } else {
        printf("The maximum number is: %d\n", num2);
    }
    return 0;
}
```

```
Enter the first number: 25
Enter the second number: 10
The maximum number is: 25
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    int result;
```

```
    // Get the number X from the user
```

```
    printf("Enter an integer (X): ");
```

```
    scanf("%d", &x);
```

```
    // Check if the number is even or odd
```

```
    if (x % 2 == 0) {
```

```
        // It's an even number
```

```
        result = 2 * x + 5;
```

```
    } else {
```

```
        // It's an odd number
```

```
        result = 2 * x - 5;
```

```
    }
```

```
    // Print the final calculated result
```

```
    printf("The result is: %d\n", result);
```

```
    return 0;
```

```
}
```

Max_x.c An algorithm that reads a number like X from the input. If the number is even, it calculates $2*X + 5$. But if it is odd, it calculates $2*X - 5$ and outputs it.

```
Enter an integer (X): 10
The result is: 25
```

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    int i = 0;  
    int number;  
    long long sum = 0;  
  
    printf("Enter the total count of numbers (N): ");  
    scanf("%d", &n);  
  
    printf("Now enter the %d numbers:\n", n);  
  
    while (i < n) {  
        scanf("%d", &number);  
        sum = sum + number;  
        i++;  
    }  
  
    printf("The final sum is: %lld\n", sum);  
  
    return 0;  
}
```

Sum.C code that reads N numbers from the input, calculates their sum, and outputs the result

```
Enter the total count of numbers (N): 4  
Now enter the 4 numbers:  
10  
5  
-2  
20  
The final sum is: 33
```

```
#include <stdio.h>
```

```
int main() {  
    int n;  
    int counter;  
    int number;  
    int max;  
    printf("How many numbers do you want to enter? ");  
    scanf("%d", &n);  
    if (n <= 0) {  
        printf("No numbers to compare.\n");  
        return 0;  
    }  
    printf("Enter number 1: ");  
    scanf("%d", &max);  
    counter = 1;  
    while (counter < n) {  
        printf("Enter number %d: ", counter + 1);  
        scanf("%d", &number);  
        if (number > max) {  
            max = number;  
        }  
        counter++;  
    }  
    printf("\nThe largest number is: %d\n", max);  
    return 0;  
}
```

Max.c Read several numbers, find the largest number, and print it.

How many numbers do you want to enter? 5

Enter number 1: 12

Enter number 2: 8

Enter number 3: 25

Enter number 4: -4

Enter number 5: 19

The largest number is: 25

Fact.C A C code that reads a positive integer N from the input and calculates its factorial

```
#include <stdio.h>
int main() {
    int n;
    unsigned long long factorial = 1;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    if (n < 0) {
        printf("Error! Factorial does not exist for negative numbers.\n");
    } else {
        int temp = n;
        while (temp > 1) {
            factorial *= temp;
            temp--;
        }
        printf("Factorial of %d = %llu\n", n, factorial);
    }
    return 0;
}
```

Enter a positive integer: 5
Factorial of 5 = 120

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int number, binary = 0, remainder, i = 1;
```

```
    scanf("%d", &number);
```

```
    while (number != 0) {
```

```
        remainder = number % 2;
```

```
        number = number / 2;
```

```
        binary = binary + (remainder * i);
```

```
        i = i * 10;
```

```
    }
```

```
    printf("%d", binary);
```

```
    return 0;
```

```
}
```

Dec2bin.c A C program that reads a decimal (base-10) number, converts it to binary (base-2), and prints the result.

Enter a decimal number: 3

Binary of 3 is 11

Press any key to continue . . .

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int number, binary = 0, remainder, i = 1;
```

```
    scanf("%d", &number);
```

```
    while (number != 0) {
```

```
        remainder = number % 2;
```

```
        number = number / 2;
```

```
        binary = binary + (remainder * i);
```

```
        i = i * 10;
```

```
    }
```

```
    printf("%d", binary);
```

```
    return 0;
```

```
}
```

Dec2bin.c A C program that reads a decimal (base-10) number, converts it to binary (base-2), and prints the result.

Enter a decimal number: 3

Binary of 3 is 11

Press any key to continue . . .


```

#include <stdio.h>
int main(void) {
    int n, reversed = 0, remainder;
    scanf("%d", &n);
    while (n != 0) {
        remainder = n % 10;
        reversed = reversed * 10 + remainder;
        n /= 10;
    }
    printf("%d", reversed);
    return 0;
}

```

reverse.c A program that reads an integer, calculates its reverse, and prints the result.

Enter an integer: 724
Reversed number = 427